Event discovery and ticketing PostgreSQL, MongoDB and Redis

- Jean Paul Collazo
- Daniel Graham
- Janeth Lucero Garcia Rodriguez



Problems

An event platform needs to manage different types of data.



Customers need to browse events and purchase tickets.



Orders and payments need reliable records.

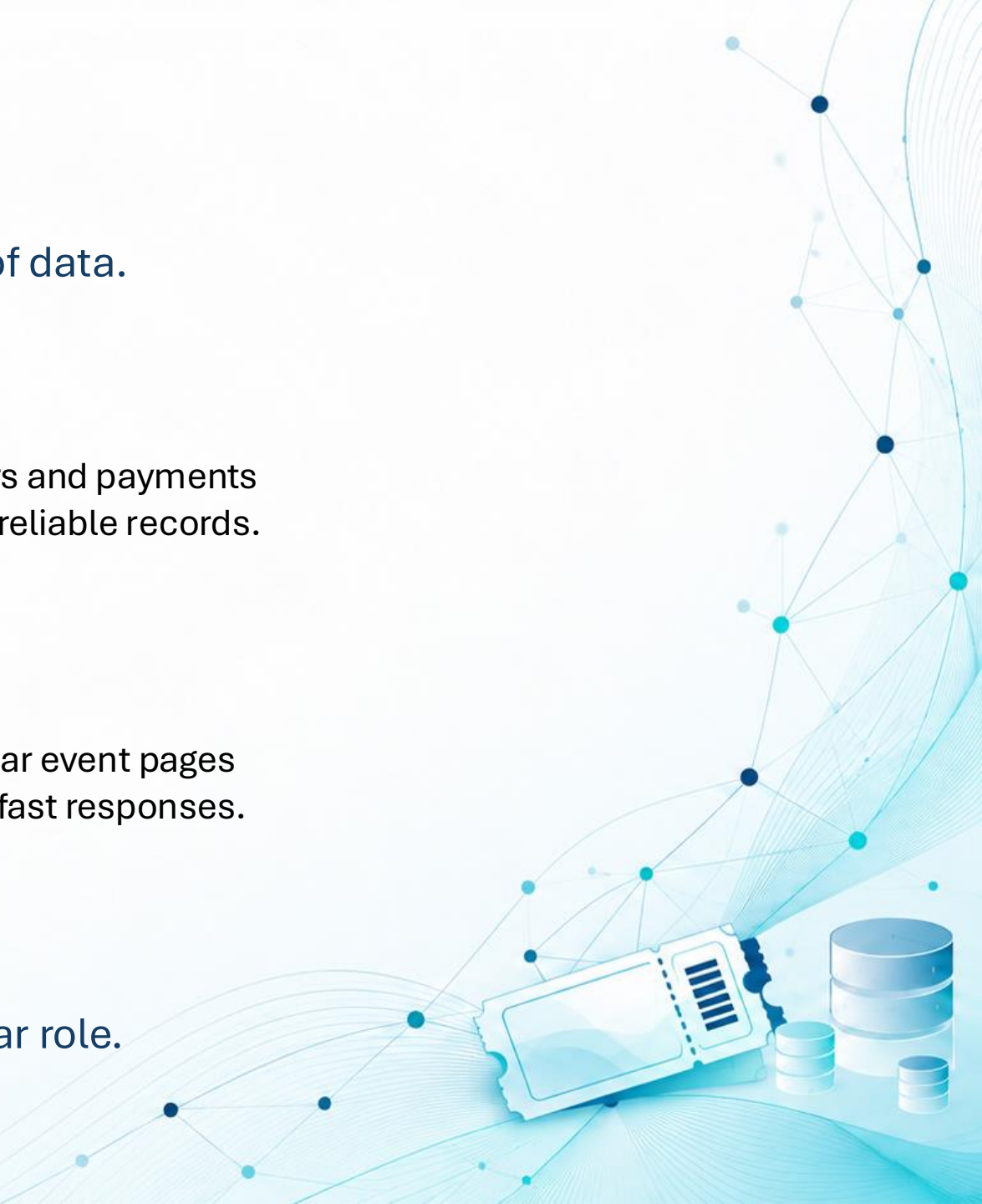


Event content varies, including speakers, schedules, and reviews.

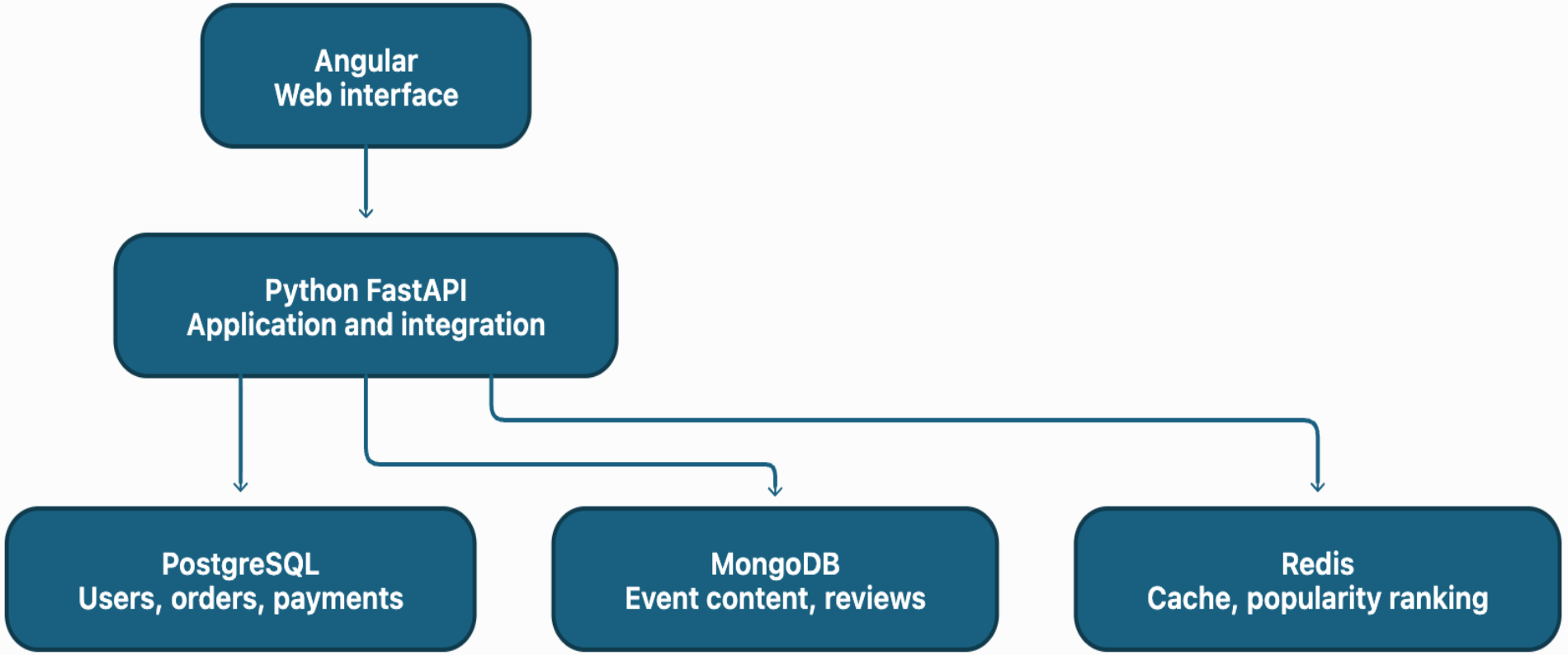


Popular event pages need fast responses.

Our goal: Connect three databases, each with a clear role.



System Architecture



Docker Compose runs all components locally.

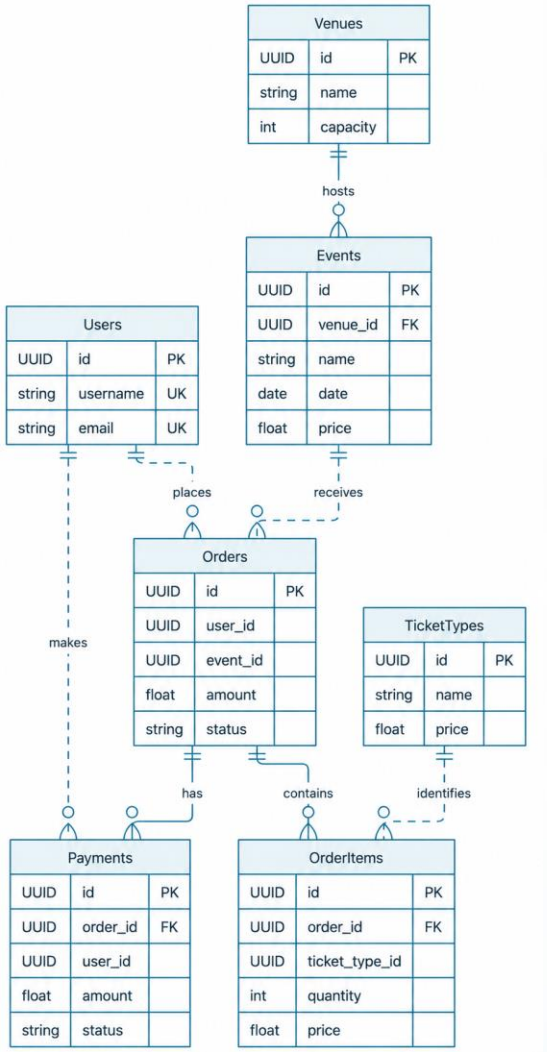
Why we used three databases

- Each database supports a different need.

Database	What it stores	Why we chooses it
PostgreSQL	Users, orders, and payments	Supports relationships and reliable transactions
MongoDB	Event descriptions, schedules, and reviews	Supports flexible fields and nested documents
Redis	Cached event responses and popularity scores	Provides fast access and event ranking



Relational Database Design



- UUID primary keys identify each record.
- Unique usernames and emails prevent duplicates.
- Foreign keys connect venues to events, and orders to items and payments.
- Indexes support searches on related IDs.

PostgreSQL: Queries and Transactions

SQL Queries	Order Transaction
Find events at a venue	1. Create the order
Find a customer's purchased tickets	2. Add order items
Calculate tickets sold and remaining inventory	**3. COMMIT** if all operations succeed
Calculate revenue by event and month	If any operation fails: ROLLBACK
Identify top customers and events above a sales threshold	One shared session keeps the order and its items together

A transaction prevents partially saved orders.



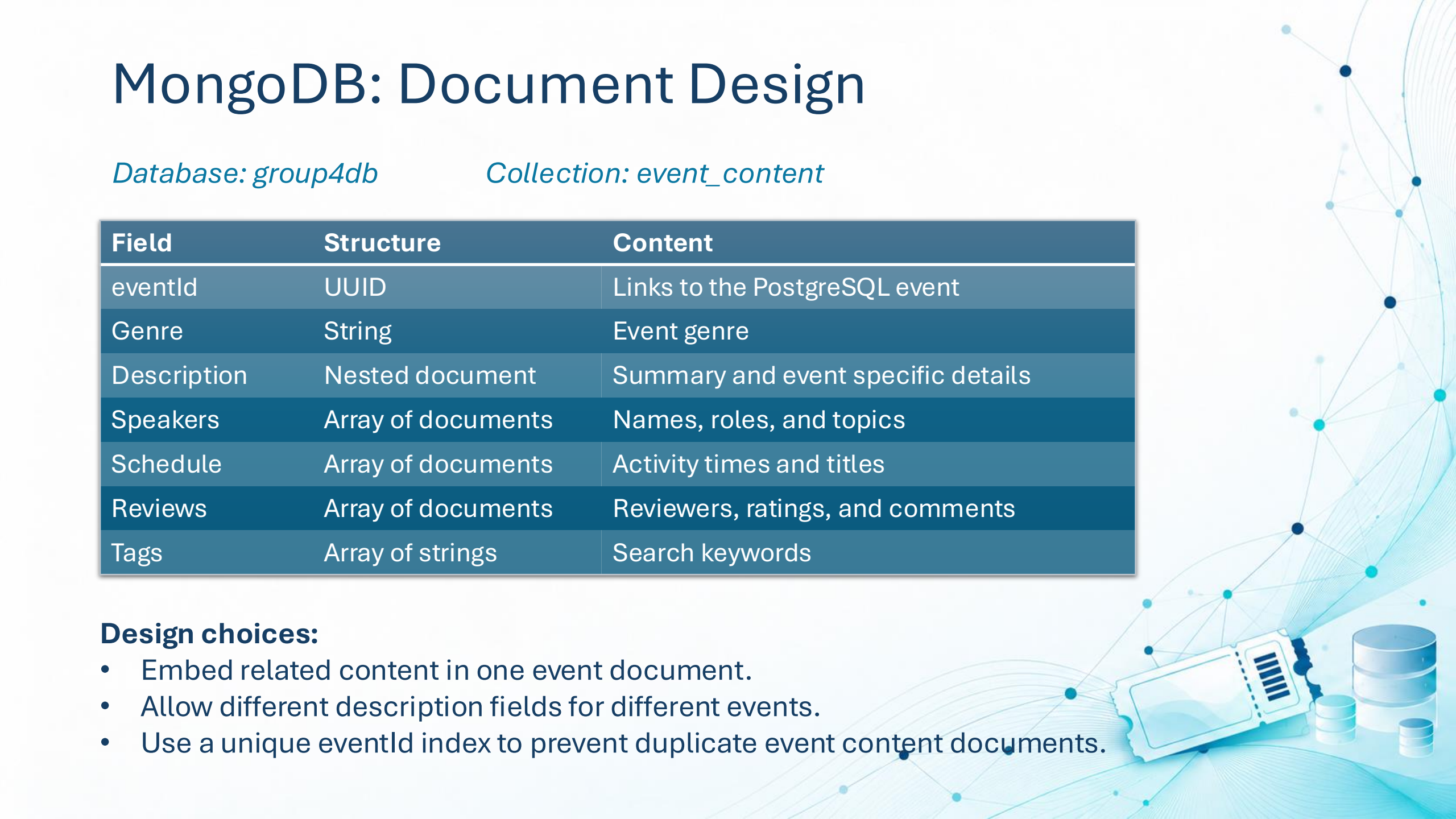
MongoDB: Document Design

Database: group4db *Collection: event_content*

Field	Structure	Content
eventId	UUID	Links to the PostgreSQL event
Genre	String	Event genre
Description	Nested document	Summary and event specific details
Speakers	Array of documents	Names, roles, and topics
Schedule	Array of documents	Activity times and titles
Reviews	Array of documents	Reviewers, ratings, and comments
Tags	Array of strings	Search keywords

Design choices:

- Embed related content in one event document.
- Allow different description fields for different events.
- Use a unique eventId index to prevent duplicate event content documents.



MongoDB: Queries and Operations

The API supports event searches and review changes.

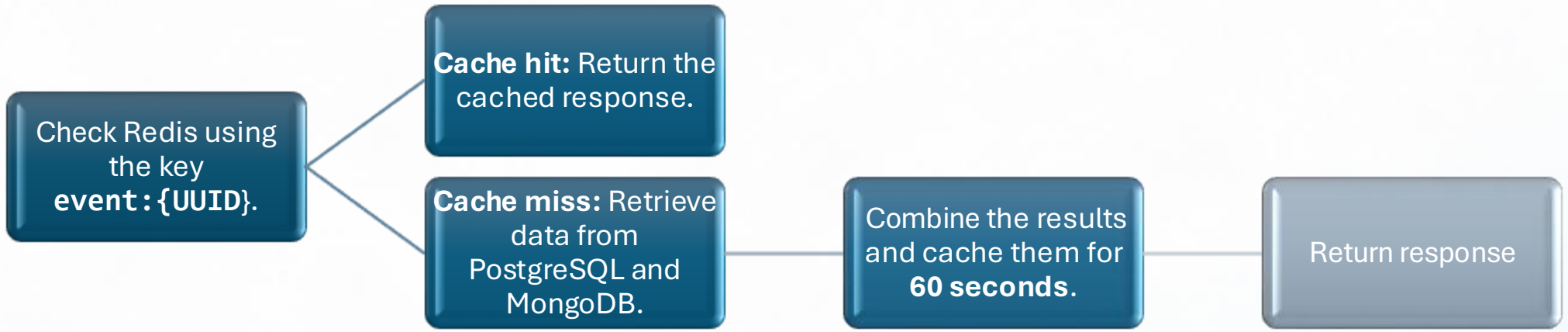
Operation	MongoDB feature	Purpose
Search by tag	Array matching	Find events with a selected tag
Search by speaker	Dot notation	Search Speakers.name
Filter reviews by rating	$gte and $elemMatch	Find reviews meeting a minimum rating
Combine search conditions	$and, $or, and $elemMatch	Match genre and nested attributes
Add a review	$push	Append a review to the event document
Remove reviews	$pull	Remove reviews matching a reviewer

Projection excludes _id from event-content responses.



Redis: Event Cache

Redis stores a temporary copy of the combined event response.



Tradeoff: Cached reviews may be outdated until expiration.

Redis: Trending Events

A sorted set ranks events by their view counts.

Key: `trending:events`

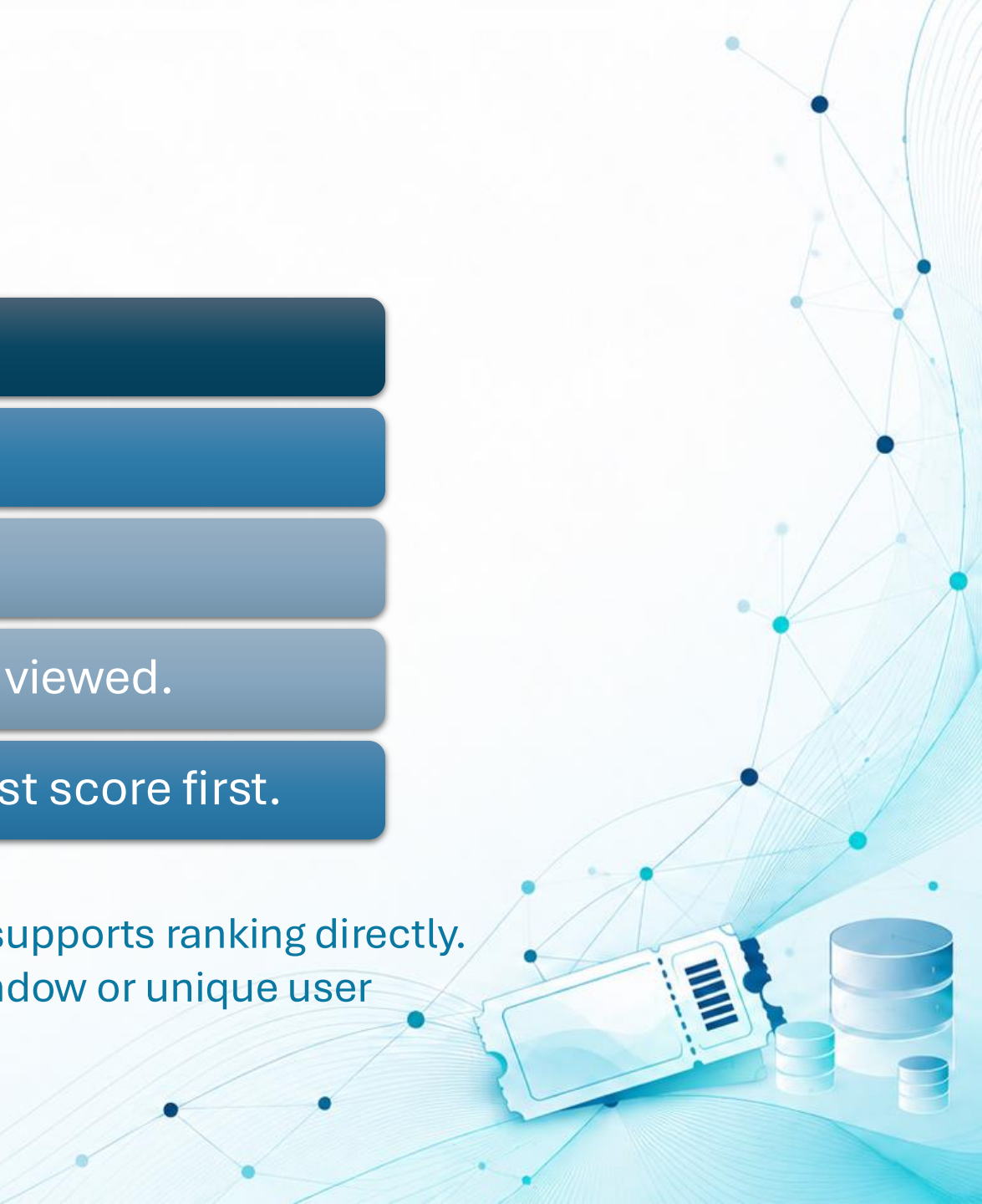
Member: Event UUID

Score: Number of counted event views

`ZINCRBY`: Adds one when an existing event is viewed.

`ZREVRANGE`: Returns the top 10 events, highest score first.

- **Why this design?** A sorted set stores scores and supports ranking directly.
- **Limitation:** Scores accumulate without a time window or unique user filter.



Cross Database Integration

One event request connects all three databases.

Example: A customer opens “Indie Sounds Live.”

Component	Role in the request
Redis	Checks for a cached event response
PostgreSQL	Retrieves the event name, date, and price on a cache miss
MongoDB	Retrieves descriptions, speakers, schedules, reviews, and tags
FastAPI	Combines the results into one response
Redis	Caches the response for 60 seconds and increases the view score

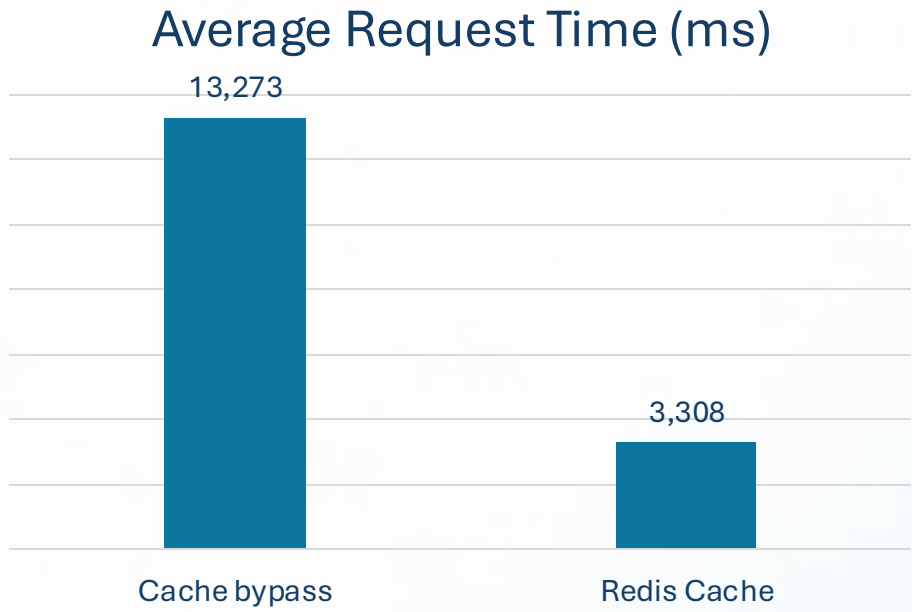
The same event UUID links the records across databases.



Performance Results

Redis caching reduced average request time.

Metric	Cache bypass	Redis cache
Minimum	11.269 ms	2.802 ms
Maximum	16.087 ms	4.339 ms
Average	13.273 ms	3.308 ms
Median	12.590 ms	3.248 ms



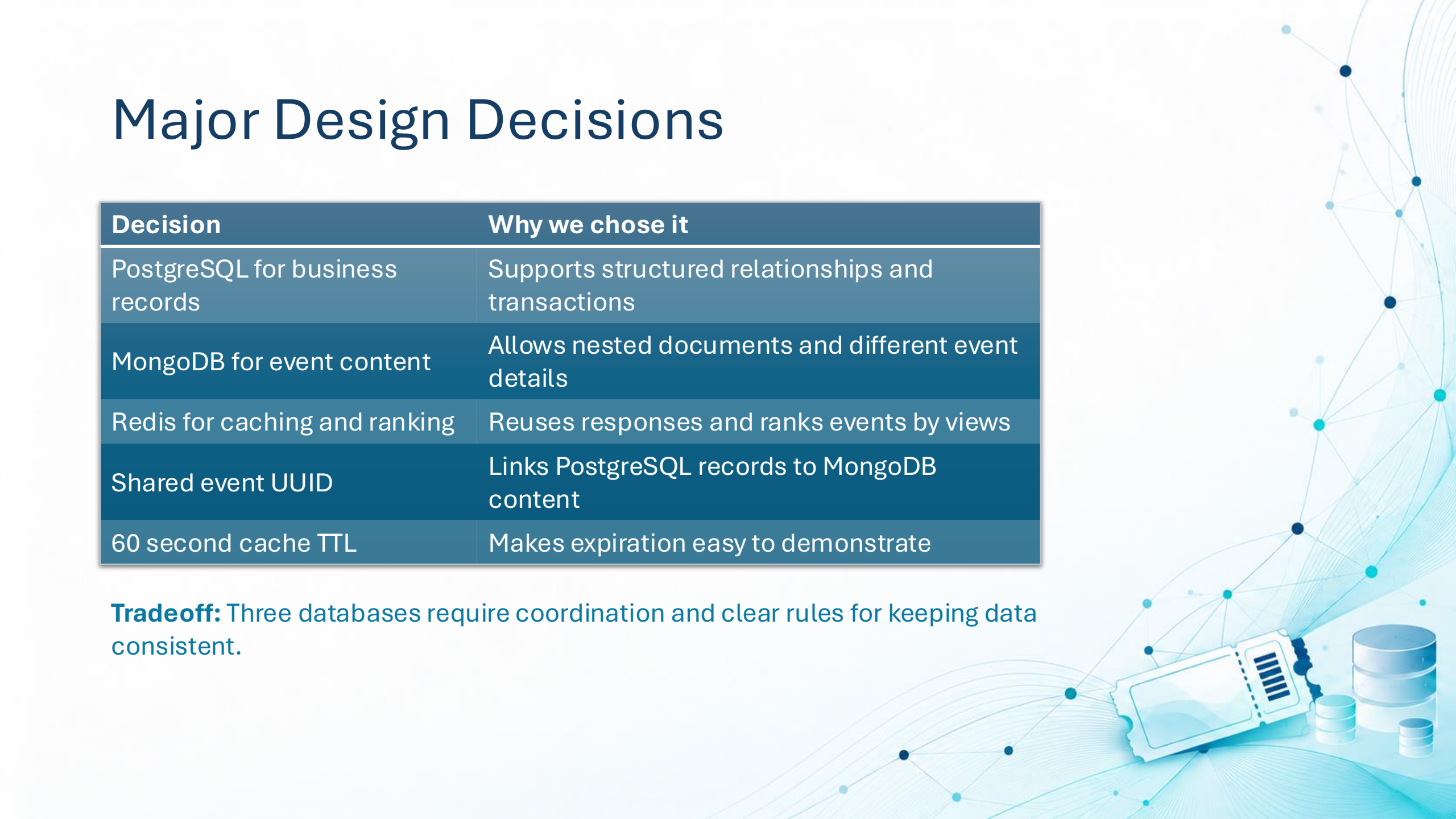
Approximately 75.1% lower average request time.

- 10 requests per case
- Cache hits avoid repeated PostgreSQL and MongoDB retrieval.
- This small experiment does not establish production performance.

Major Design Decisions

Decision	Why we chose it
PostgreSQL for business records	Supports structured relationships and transactions
MongoDB for event content	Allows nested documents and different event details
Redis for caching and ranking	Reuses responses and ranks events by views
Shared event UUID	Links PostgreSQL records to MongoDB content
60 second cache TTL	Makes expiration easy to demonstrate

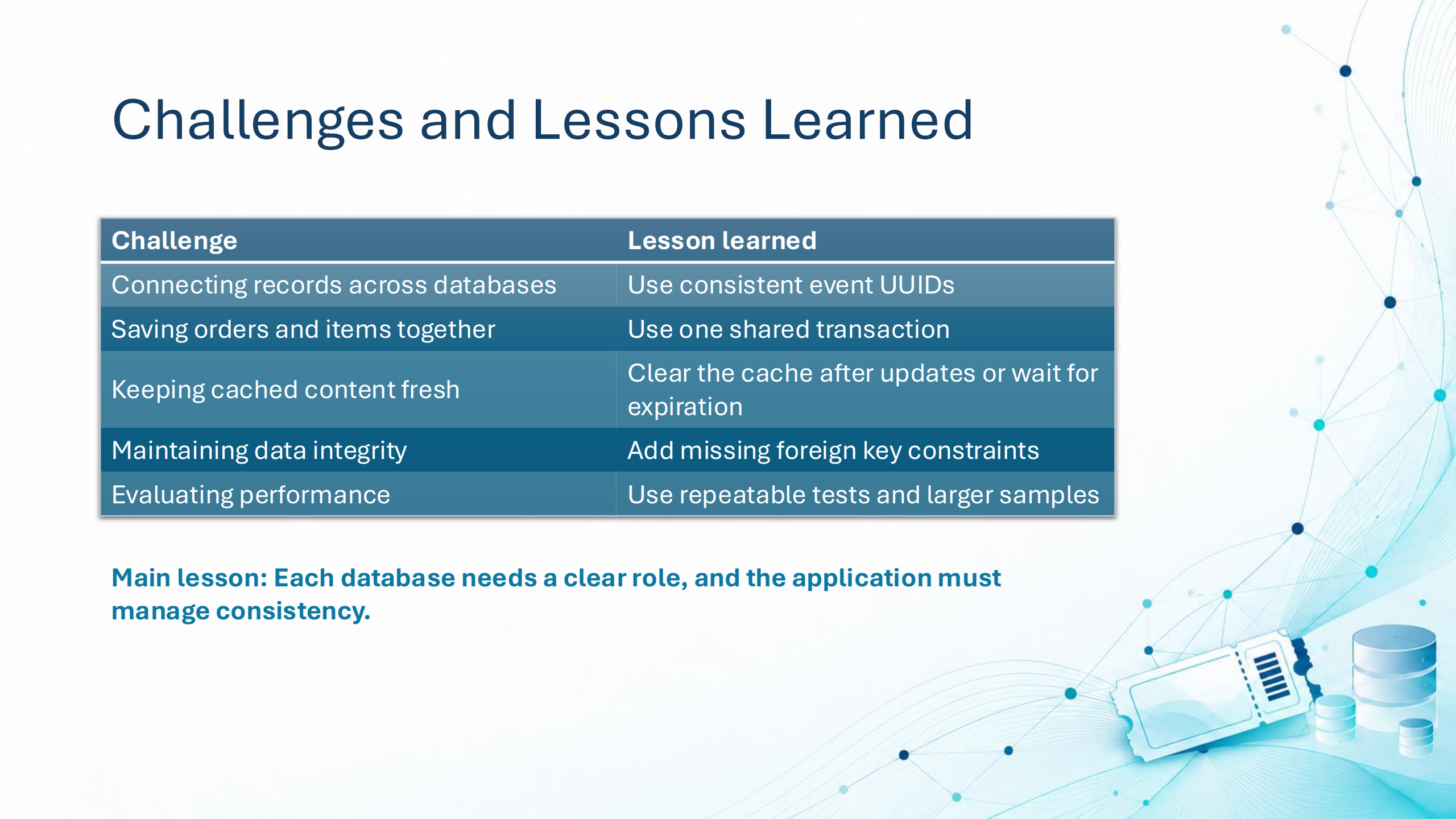
Tradeoff: Three databases require coordination and clear rules for keeping data consistent.



Challenges and Lessons Learned

Challenge	Lesson learned
Connecting records across databases	Use consistent event UUIDs
Saving orders and items together	Use one shared transaction
Keeping cached content fresh	Clear the cache after updates or wait for expiration
Maintaining data integrity	Add missing foreign key constraints
Evaluating performance	Use repeatable tests and larger samples

Main lesson: Each database needs a clear role, and the application must manage consistency.



If We Used Only One Database

We would choose PostgreSQL.

WHAT BECOMES EASIER



WHAT REQUIRES A DIFFERENT APPROACH



Conclusions

Demo

- **PostgreSQL** supports structured records and order transactions.
- **MongoDB** supports flexible and nested event content.
- **Redis** provides event caching and popularity ranking.
- The reported experiment showed **75.1% lower average request time** with caching.
- Integrating three databases requires clear data ownership and consistency rules.

